Team 1 — Backend (Node.js) Production Roadmap

Async Pipeline · Security · Rate Limiting · Webhooks · Observability

Backend Team | YourCrawl Dark Pattern Auditor | Generated: 16 August 2026

🔧 YourCrawl — Team 1: Backend (Node.js) Production Roadmap

Assigned To: You (Product Manager / Backend Lead)

Priority: CRITICAL — Core orchestration layer. All other teams depend on this.

🔍 What Exists Today (Current State Assessment)

Component	File	Status
Express Server	`server.js`, `src/app/app.js`	**Functional**
Puppeteer Crawler	`src/services/crawl.service.js`	Works, but synchronous + blocking
Gemini AI Report Gen	`src/services/ai.service.js`	Works
Auth System	`src/controllers/auth.controller.js`	JWT + Google OAuth present
Audit Persistence	`src/models/auditReport.model.js`	MongoDB Atlas present
RAG Service Client	`src/services/rag.service.js`	Functional
Mail Service	`src/services/mail.service.js`	Exists but unused
Rate Limiting	None	MISSING
Job Queue	None	CRITICAL GAP
Usage Metering	None	MISSING
WebSocket / SSE	None	MISSING
Webhook System	None	MISSING
Secrets Management	`.env` with hardcoded keys	SECURITY RISK
Screenshot Storage	Local `/screenshots` folder	NOT SCALABLE

Critical Flaws Found

1. Fully Synchronous Audit Pipeline (Showstopper)

File: `src/services/ai.service.js` — `runCrawlAgent()` The entire pipeline Puppeteer crawl to ML scan to Gemini report runs synchronously inside a single HTTP request. For a 3-page audit, this takes 60-180 seconds. The HTTP connection will time out, the user sees nothing, and all work is lost.

```
POST /api/crawl → [Puppeteer] → [ML service] → [Gemini] → Response
                                  (2-3 minutes!!!)
```

Fix: Implement a Job Queue (Feature 1 below).

---

2. Secrets Exposed in `.env` (Security Risk)

File: `Backend/.env` The `.env` file contains real credentials: MongoDB Atlas URI with username/password, Gemini API key, Google OAuth secrets, JWT secret, Gmail app password. This file is likely committed to Git.

Immediate Actions:

- Rotate ALL credentials right now
- Add `.env` to `.gitignore`
- Run `git rm --cached .env` if it was ever committed
- Move to proper secrets management for production (AWS Secrets Manager / Doppler)

---

3. No Rate Limiting / Abuse Protection

Any user or bot can hammer `POST /api/crawl` and rack up thousands of Gemini API calls (costly), hundreds of concurrent Puppeteer instances (server crash), and unlimited MongoDB writes.

---

4. Local Screenshot Storage (Not Production-Ready)

File: `crawl.service.js` Line ~727 Screenshots are saved to `./screenshots/` on the server disk. This blocks stateless/auto-scaling deployments, fills up the disk, and is not accessible via CDN. The `imagekit` dependency is installed but not wired up.

---

5. In-Memory Async Scan Store (ML Service State Loss)

File: `ML/api.py` Line 125

```
scan_store: dict[str, ScanStatusResponse] = {}
```

The ML service uses a Python in-memory dict for async scan results. On restart, all in-flight scans are permanently lost.

---

6. No User Usage Limits / Subscription Tiers

Any registered user can run unlimited audits for free forever. No free tier, no pro tier, no credit system, no payment integration.

---

7. No Webhook / CI/CD Integration Support

Enterprise customers need to plug audits into GitHub Actions, Jira, Slack, etc. Currently impossible.

System Architecture: Target State

```
Client
  |
  v
[Express API] --auth--> [JWT Middleware]
  |
  +-- POST /api/crawl -----------------> [Bull Job Queue (Redis)]
  |                                              |
  +-- GET  /api/crawl/status/:jobId <------------+
  |                                              |
  +-- SSE /api/crawl/stream <--------------------+


         [Bull Worker Process]
              |
              +-- Puppeteer Crawl
              |         |-- screenshot --> [ImageKit / S3 CDN]
              |
              +-- ML Service  (http://ml:8000)
              |
              +-- Gemini AI Report
              |
              +-- MongoDB Save --> Notify Client via SSE


[Redis]       <->  Bull Queue + Rate Limiter
[MongoDB Atlas] <->  Users, AuditReports, UsageLogs
```

Feature Tasks to Build

Feature 1 — Async Job Queue with Real-Time Progress (HIGHEST PRIORITY)

Why: The current synchronous pipeline is a showstopper. No user will wait 3 minutes with zero feedback.
Technology: `bull` (Redis-backed job queue) + Server-Sent Events (SSE) **Implementation Steps:**

5. Install dependencies: `npm install bull ioredis`

6. Create `src/queues/auditQueue.js`:

```
import Bull from 'bull';
export const auditQueue = new Bull('audit', {
  redis: { host: process.env.REDIS_HOST, port: 6379 }
});
```

7. Create `src/workers/auditWorker.js` that:

- Processes jobs from the queue - Reports progress: 25% after crawl, 50% after ML, 75% after AI, 100% complete - Saves result to MongoDB - Emits completion event

8. Modify `crawlUrl` controller to enqueue a job instead of awaiting `runCrawlAgent`. Return `{ jobId, status: 'queued' }` with HTTP 202 immediately.
9. Add SSE endpoint `GET /api/crawl/status/stream` — client connects, backend streams progress events.
10. Add `GET /api/crawl/jobs/:jobId` for polling fallback.

Acceptance Criteria: A crawl request returns HTTP 202 with a jobId in under 1 second. The frontend receives live progress events.

Feature 2 — Screenshot Upload to ImageKit/S3

Why: Local disk storage is not scalable and blocks stateless deployments. `imagekit` is already installed but not wired up. **Implementation Steps:**

11. Create `src/services/storage.service.js`:

```
import ImageKit from 'imagekit';
const ik = new ImageKit({
  publicKey: config.imagekitPublicKey,
  privateKey: config.imagekitPrivateKey,
  urlEndpoint: config.imagekitUrlEndpoint,
});
export const uploadScreenshot = async (buffer, filename) => {
  const result = await ik.upload({
    file: buffer,
    fileName: filename,
    folder: '/yourcrawl/screenshots'
  });
  return result.url;
};
```

12. In `crawl.service.js`, replace `fs.writeFileSync(screenshotPath, ...)` with a call to `uploadScreenshot()`.
13. Store the returned CDN URL in MongoDB's `auditData.screenshot_url`.
14. Add `IMAGEKIT_PUBLIC_KEY`, `IMAGEKIT_PRIVATE_KEY`, `IMAGEKIT_URL_ENDPOINT` to `.env`.

Acceptance Criteria: After a crawl, screenshot is accessible via HTTPS CDN URL, not a local file path.

Feature 3 — Rate Limiting and Abuse Protection

Implementation Steps:

15. Install: `npm install express-rate-limit rate-limit-redis`
16. Create `src/middlewares/rateLimiter.middleware.js`:

```
import rateLimit from 'express-rate-limit';


export const globalLimiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 100 });


export const crawlLimiter = rateLimit({
  windowMs: 60 * 60 * 1000,
  max: 10,
  keyGenerator: (req) => req.user?.id || req.ip,
  message: { error: 'Audit limit reached. Upgrade your plan for more audits.' }
});
```

17. Apply `globalLimiter` in `app.js` and `crawlLimiter` on the crawl route.

Acceptance Criteria: A single user cannot trigger more than 10 audits per hour.

Feature 4 — Usage Metering and Subscription Tiers

Why: Required for monetization. Without this, the product cannot charge users. **Implementation Steps:**

18. Add `src/models/usageLog.model.js`: `{ userId, action: 'audit', url, status, createdAt }`
19. Add usage fields to `user.model.js`:

```
plan: { type: String, enum: ['free', 'pro', 'enterprise'], default: 'free' },
auditsThisMonth: { type: Number, default: 0 },
auditsLimit: { type: Number, default: 5 },
planResetDate: { type: Date },
```

20. Create `src/middlewares/usageGuard.middleware.js` that checks if `user.auditsThisMonth >= user.auditsLimit` and returns HTTP 402 with an upgrade message.
21. Log usage in the audit worker after each successful audit.
22. Add `GET /api/user/usage` endpoint.

Acceptance Criteria: A free-tier user sees an upgrade prompt after 5 audits/month.

Feature 5 — Webhook Support for CI/CD Integration

Implementation Steps:

23. Add `src/models/webhook.model.js`: `{ userId, targetUrl, events, secret, active }`
24. Add CRUD endpoints: `POST /api/webhooks`, `GET /api/webhooks`, `DELETE /api/webhooks/:id`

25. In the audit worker on completion, find all active webhooks for the user and POST the audit result with an HMAC signature header.

Acceptance Criteria: After an audit completes, the configured webhook URL receives a POST request within 5 seconds.

Feature 6 — Persistent Async Scan Store (Fix ML In-Memory Issue)

Implementation Steps: Replace the Python dict in `ML/api.py` with Redis:

```
import redis, json

r = redis.Redis(host=os.getenv('REDIS_HOST', 'localhost'))


def save_scan(scan_id, data):

    r.setex(f"scan:{scan_id}", 3600, json.dumps(data))


def get_scan(scan_id):

    val = r.get(f"scan:{scan_id}")

    return json.loads(val) if val else None
```

Replace all `scan_store[scan_id]` references with `save_scan()` / `get_scan()`. **Acceptance Criteria:** If the ML service restarts mid-scan, the result is recoverable from Redis.

Feature 7 — Health Checks and Observability

Implementation Steps:

26. Add `GET /health` endpoint that checks MongoDB, Redis, ML Service, and RAG Service connectivity.
27. Add structured logging with `pino` : `npm install pino pino-http`. Replace `console.log` with `logger.info({ userId, url, duration })`.
28. Add request ID middleware — attach a unique `X-Request-ID` to every request.

Acceptance Criteria: `/health` returns JSON with per-service status. All requests have a request ID in logs.

Feature 8 — Email Notifications (Wire Up mail.service.js)

Why: `mail.service.js` and `mail.templates.js` already exist but are never called. **Implementation Steps:** After a successful audit in the job worker:

```
import { sendAuditCompleteEmail } from '../services/mail.service.js';

await sendAuditCompleteEmail(user.email, { url, reportId, riskScore, findings });
```

Add "Audit Failed" email template and send on job failure. **Acceptance Criteria:** Users receive an email with a link to their report within 1 minute of audit completion.

Security Hardening Checklist

Task	Priority
Rotate ALL credentials in `.env` immediately	**CRITICAL**
Add `.env` to `.gitignore`, run `git rm --cached .env`	CRITICAL
Move secrets to environment variables / secrets manager	CRITICAL
Add `helmet.js` for security headers	HIGH
Validate all incoming webhook URLs (block SSRF)	HIGH
Add CORS origin whitelist (not `*`)	HIGH
Add input sanitization on URL field	HIGH

Delivery Timeline

Week	Milestone
Week 1	**Feature 1 (Job Queue + SSE) + Feature 2 (Screenshot CDN)**
Week 2	Feature 3 (Rate Limiting) + Feature 4 (Usage Metering)
Week 3	Feature 5 (Webhooks) + Feature 7 (Health Checks)
Week 4	Feature 6 (Redis Scan Store) + Feature 8 (Email) + Security Hardening

New Dependencies to Add

```
bull ^4.x
ioredis ^5.x
express-rate-limit ^7.x
pino ^9.x
pino-http ^10.x
helmet ^8.x
```

Definition of "Production Ready" for Backend

- [ ] No audit request blocks the HTTP connection for more than 2 seconds
- [ ] Screenshots stored on CDN, not local disk
- [ ] All secrets in environment variables / secrets manager
- [ ] Rate limiting active on all endpoints
- [ ] Usage metering enforced per user plan

- [ ] Health check endpoint returns status of all dependent services
- [ ] Structured JSON logs with request IDs
- [ ] Webhook delivery working
- [ ] Email notifications on audit complete/fail
- [ ] Zero raw credentials in source code or Git history
